AUTONOMOUS DRONE FLEET COORDINATION SIMULATOR

Semester Project Report

Difficulty Level: Full

Suggested Programming Language: Java

Course Name:

CS – 212: Object Oriented Programming

Submitted By:

Hassan Ali(512606), Hamza Mudassir(504982), M. Soban Hussain(505247)

Instructor Name:

Mam Anum, LE Mam Hira

Institution Name:

Department Of Computer and Software Engineering

College Of Electrical and Mechanical Engineering, Rawalpindi

Submission Date:

22nd,Dec,2025

Contents

Table of Contents .....
1. Introduction 3
2. Problem Statement.....3
3. Objectives 3
3.1 Primary Objective 3
3.2 Secondary Objectives 3
3.3 Learning Outcomes 4
4. System Overview 4
5. System Design 4
5.1 UML Class Diagram.....4
5.2 System-Level Architecture 6
6. Mathematical and CEP Modeling 6
6.1 Translational Dynamics.....
6.2 Rotational Dynamics.....
6.3 Control Logic .....
6.4 Formation and Collision Avoidance.....
6.5 Communication Modeling .....
7. Implementation Details.....6
8. Key Features.....7
9. Results and Outputs.....7
10. Challenges and Solutions.....11
11. Conclusion 11
12. Future Enhancements 11
13. References.....11

1. Introduction

The rapid advancement of autonomous cyber-physical systems has led to the widespread deployment of drone fleets in applications such as precision agriculture, surveillance, mapping, disaster management, and environmental monitoring. Coordinating multiple drones in a shared environment is a complex task due to dynamic constraints, safety requirements, communication limitations, and control stability issues. Testing such systems directly on hardware is expensive, risky, and difficult to reproduce.

Simulation therefore plays a critical role in evaluating drone coordination algorithms before real-world deployment. This project presents an **Autonomous Drone Fleet Coordination Simulator**, designed to model the collective behavior of multiple quadrotor drones operating in a three-dimensional environment. Each drone is implemented as an independent autonomous agent capable of navigation, communication, collision avoidance, and formation control.

The simulator demonstrates how object-oriented programming principles such as encapsulation, modularity, and composition can be applied to design a scalable cyber-physical simulation platform.

2. Problem Statement

Real-world implementation of coordinated multi-drone systems faces several challenges:

- High risk of collisions during testing
- Limited observability of internal states
- Difficulty in reproducing scenarios
- High hardware and maintenance costs

Manual coordination approaches are not scalable, while direct hardware testing can lead to system failure or damage. The problem addressed in this project is the development of a **simulation-based solution** that realistically models drone dynamics, control, communication, and coordination, enabling safe experimentation and quantitative performance evaluation.

3. Objectives

3.1 Primary Objective

To design and implement a simulation platform for autonomous coordination of multiple drones in a shared 3D environment.

3.2 Secondary Objectives

Following are the some of the secondary objectives that are mentioned below

- Implement realistic translational and rotational drone dynamics
- Design a control system for target tracking
- Enable formation control and inter-drone coordination
- Implement collision avoidance using repulsive forces
- Simulate communication constraints and packet loss
- Log system behavior and evaluate performance

3.3 Learning Outcomes

Following were the learning outcomes of this complex engineering problem.

- Object-oriented system design
- Cyber-physical system modeling
- Numerical simulation
- Control systems implementation
- Multi-agent coordination

4. System Overview

The simulator consists of multiple interacting subsystems:

- **Inputs:** Simulation constants, drone initial states, target positions
- **Processing:** Control computation, force application, motion integration
- **Outputs:** CSV logs, metrics report, GUI visualization

Each drone autonomously computes its behavior based on local and communicated information, making the system decentralized and scalable.

5. System Design

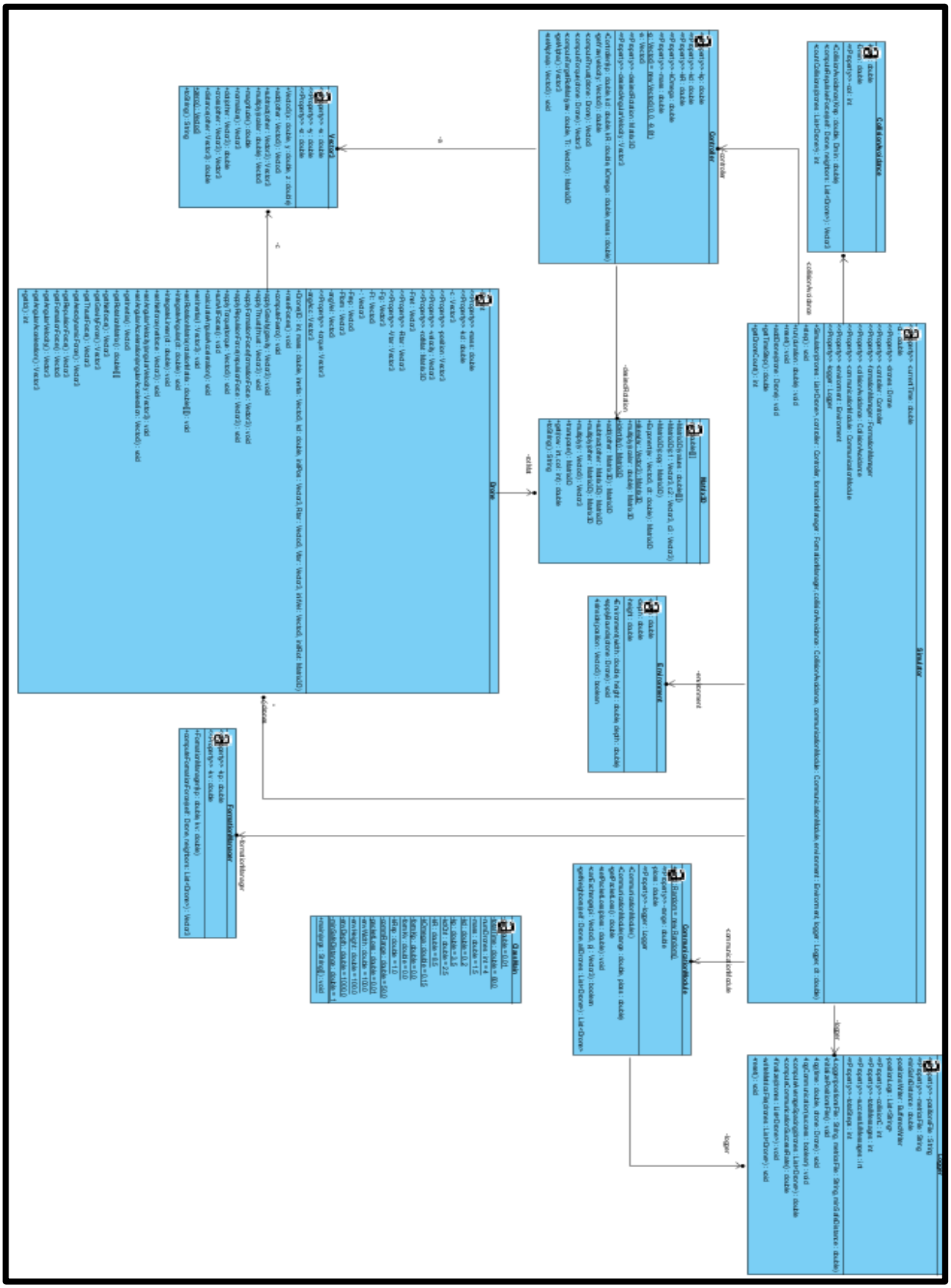
The system follows a modular object-oriented architecture. Each physical or logical component is encapsulated within a dedicated class, ensuring maintainability and extensibility.

5.1 UML Class Diagram

The UML class diagram illustrates the structural organization of the simulator. Major classes include Simulator, Drone, Controller, FormationManager, CollisionAvoidance, CommunicationModule, Environment, Logger, Vector3, and Matrix3D.

The Drone class aggregates physical state variables, while the Simulator class coordinates the execution of all subsystems. Utility classes such as Vector3 and Matrix3D provide mathematical abstractions for three-dimensional computations.

UML:



5.2 System-Level Architecture

At the system level, the Simulator orchestrates execution. During each timestep:

1. Control forces are computed
2. Drone dynamics are updated
3. Formation and collision forces are applied
4. Communication is simulated
5. Environment constraints are enforced
6. Data is logged

6. Mathematical and CEP Modeling

Physical Assumptions

- Drones are rigid bodies
- Forces act at the center of mass
- Aerodynamic drag is linear with velocity

Translational Dynamics

$$m\mathbf{a} = m\mathbf{g} + R\mathbf{T} + \mathbf{F}_{aero} + \mathbf{F}_{rep} + \mathbf{F}_{form}$$

Rotational Dynamics

$$\dot{\omega} = I^{-1}(\tau - \omega \times (I\omega))$$

Control Logic (Controller Class)

A PD control law computes desired acceleration:

$$\mathbf{a}_{cmd} = k_p(\mathbf{p}_{des} - \mathbf{p}) + k_d(\mathbf{v}_{des} - \mathbf{v}) + \mathbf{g}$$

Torque is computed to align the drone's orientation with the desired rotation matrix.

Formation Control

Consensus-based formation force:

$$\mathbf{F}_{form,i} = -\sum_{j \in N_i} \left[k_p(\mathbf{p}_i - \mathbf{p}_j) + k_v(\mathbf{v}_i - \mathbf{v}_j)\right]$$

Collision Avoidance
Repulsive potential field:

$$\mathbf{F}_{rep} = \frac{k_{rep}(\mathbf{p}_i - \mathbf{p}_j)}{\| \mathbf{p}_i - \mathbf{p}_j \|^2}$$

Communication Model
Communication succeeds if:

$$\| \mathbf{p}_i - \mathbf{p}_j \| < R_{comm} \text{and} rand() > p_{loss}$$

7. Implementation Details

Following given are the implementation details of the

- **Language:** Java
- **GUI:** JavaFX
- **Numerical Integration:** Discrete-time Euler
- **Logging:** CSV and TXT files
- **Architecture:** Fully object-oriented

The project structure cleanly separates physics, control, communication, visualization, and logging.

8. Key Features

Here are the features of our autonomous drone fleet system.

- Autonomous multi-agent drones
- Target tracking
- Formation control
- Collision avoidance
- Communication constraints
- Real-time visualization
- Performance logging
- Scalable design

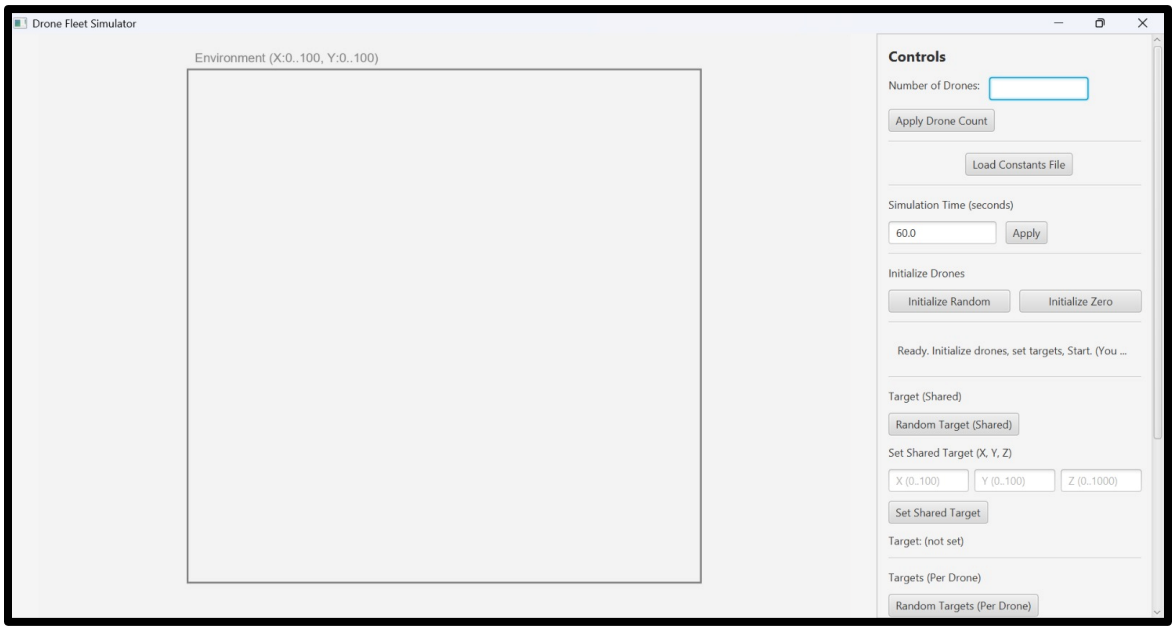
9. Results and Outputs

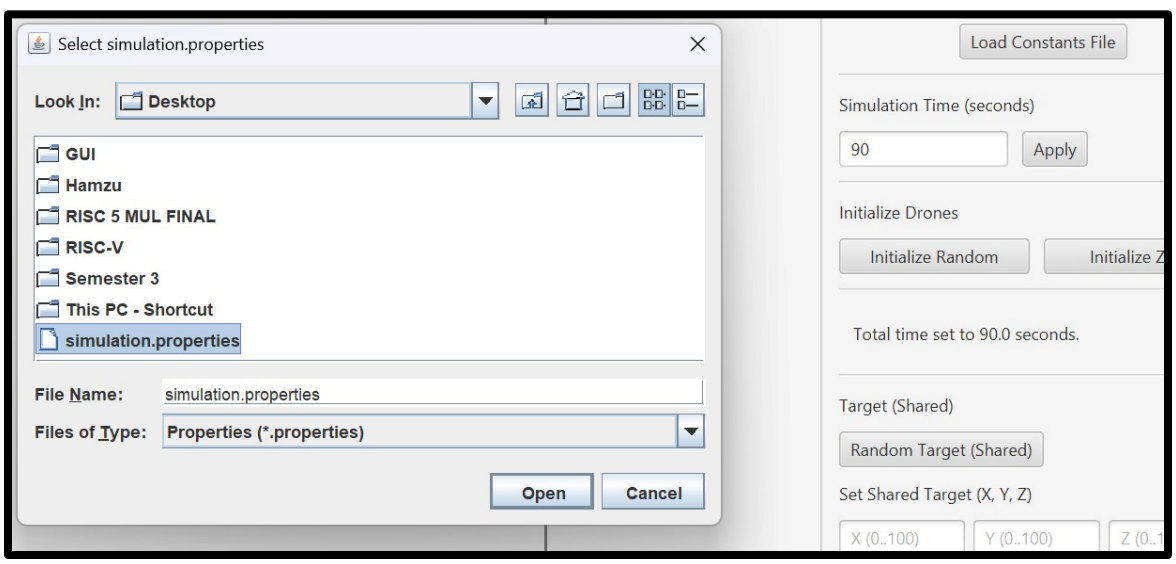
Here is how results and outputs computed:

- Positions.csv records time-step data
- Metrics.txt summarizes system performance
- GUI shows real-time drone motion

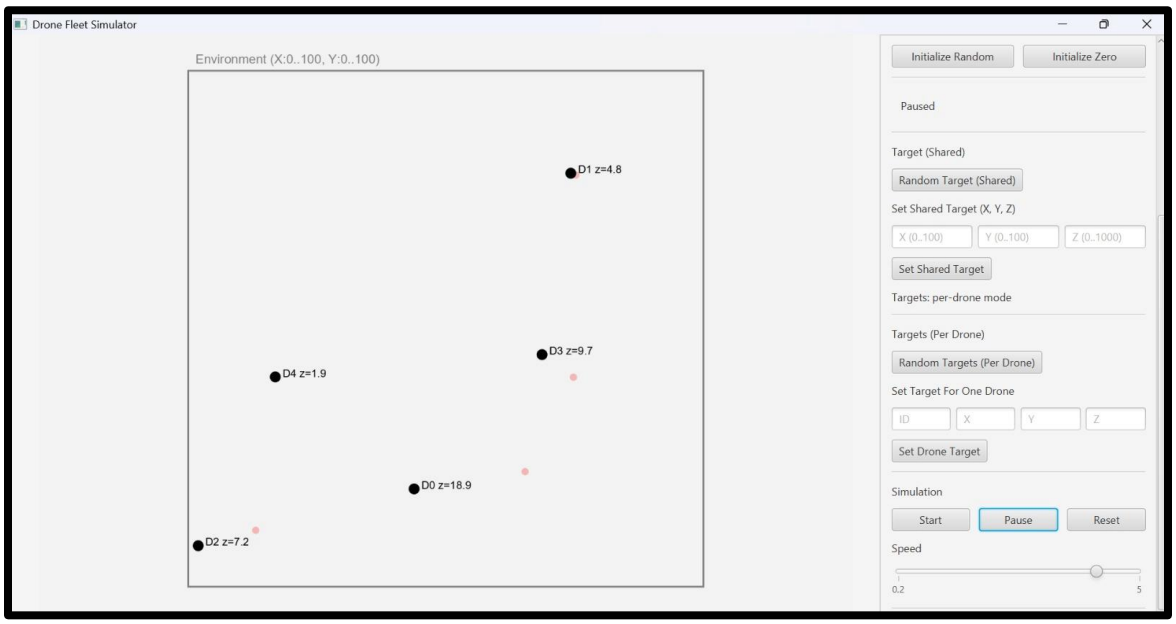
Observed results show stable convergence, safe spacing, and realistic communication effects.

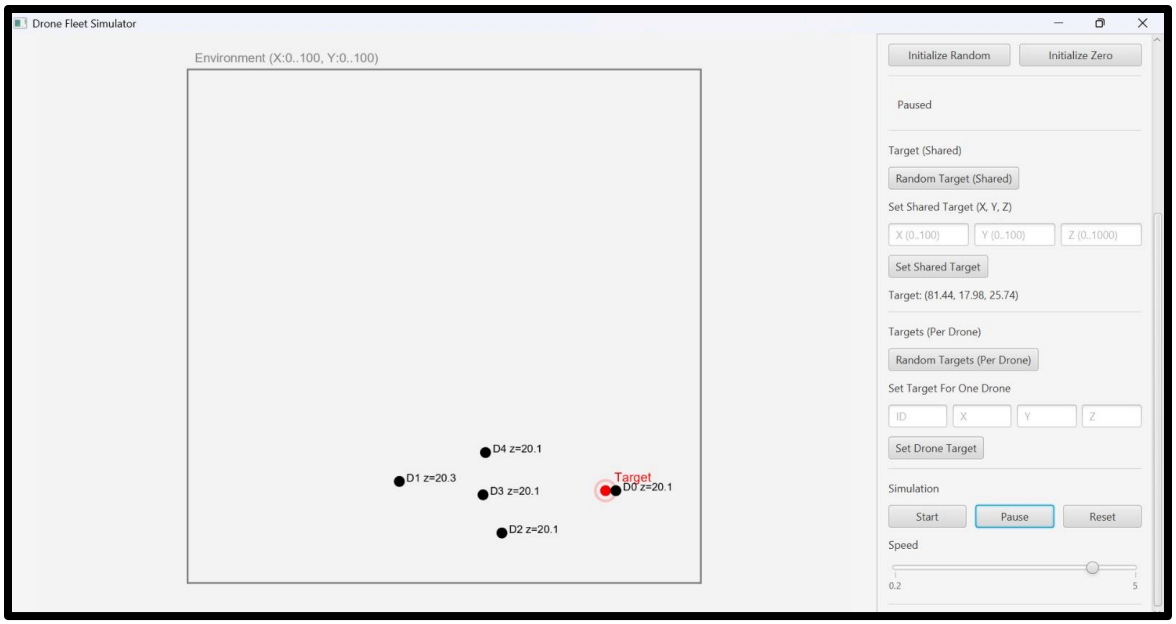
Following given are some of GUI CONSOLE outputs:





Drone Fleet Simulator
Environment (X:0..100, Y:0..100)
Target
D4 z=1.0
D2 z=1.0
Apply Drone Count
Load Constants File
Simulation Time (seconds)
90
Apply
Initialize Drones
Initialize Random
Initialize Zero
Random shared target set. Press Start.
Target (Shared)
Random Target (Shared)
Set Shared Target (X, Y, Z)
X (0..100)
Y (0..100)
Z (0..1000)
Set Shared Target
Target: (55.72, 33.00, 24.02)
Targets (Per Drone)
Random Targets (Per Drone)
Set Target For One Drone
ID
X
Y
Z


Drone Fleet Simulator
Environment (X:0..100, Y:0..100)
D1 z=4.8
D3 z=9.7
D4 z=1.9
D0 z=18.9
D2 z=7.2
Initialize Random
Initialize Zero
Paused
Target (Shared)
Random Target (Shared)
Set Shared Target (X, Y, Z)
X (0..100)
Y (0..100)
Z (0..1000)
Set Shared Target
Targets: per-drone mode
Targets (Per Drone)
Random Targets (Per Drone)
Set Target For One Drone
ID
X
Y
Z
Set Drone Target
Simulation
Start
Pause
Reset
Speed
0.2
5



Following given are some of CLI CONSOLE outputs:

```
Drone Simulation Project

Initializing 4 drones...
  All drones starting at: (0.0, 0.0, 0.0)

Enter initial position:   Drone 0: Target=(20.00, 10.00, 400.00)
  Drone 1: Target=(40.00, 10.00, 400.00)
  Drone 2: Target=(60.00, 10.00, 400.00)
  Drone 3: Target=(80.00, 10.00, 400.00)

Starting simulation...
Duration: 1.0s
Time step: 0.01s
Total steps: 100

Simulation Start
Duration: 1.0s, Time step: 0.01s
Total steps: 100
Number of drones: 4
```

```
Logging completed:
Positions saved to: Positions.csv
Metrics saved to: Metrics.txt
Simulation complete!
Final time: 59.99999999999663s

✓ Simulation completed successfully!

=== Final Drone States ===
Drone 0: Pos=(20.00, 10.00, 394.34), Vel=(0.00, 0.00, -0.00), Target=(20.00, 10.00, 400.00), Distance=5.66 m
Drone 1: Pos=(40.00, 10.00, 394.34), Vel=(-0.00, -0.00, -0.00), Target=(40.00, 10.00, 400.00), Distance=5.66 m
Drone 2: Pos=(60.00, 10.00, 394.34), Vel=(0.00, 0.00, -0.00), Target=(60.00, 10.00, 400.00), Distance=5.66 m
Drone 3: Pos=(80.00, 10.00, 394.34), Vel=(-0.00, 0.00, 0.00), Target=(80.00, 10.00, 400.00), Distance=5.66 m
```

10. Challenges and Solutions

Key challenges included numerical stability, synchronization of multiple agents, realistic communication modeling, and maintaining modularity. These challenges were addressed through careful time-step selection, modular class design, and probabilistic communication modeling.

Challenge	Solution
Numerical instability	Small timestep (dt)
Drone collisions	Repulsive forces
Communication realism	Probabilistic packet loss
System complexity	Modular OOP design

11. Conclusion

The Autonomous Drone Fleet Coordination Simulator successfully implements a realistic cyber-physical system using object-oriented design. All project objectives are achieved, demonstrating effective control, coordination, and simulation of multi-agent drone systems.

12. Future Enhancements

Future enhancements may include obstacle modeling, advanced control strategies such as MPC, sensor noise modeling, hardware-in-the-loop testing, and integration with real UAV platforms.

- Obstacle modeling
- 3D Visualization
- Sensor noise simulation
- Hardware-in-the-loop testing

13. References

1. Beard, R., & McLain, T., Small Unmanned Aircraft Systems, Princeton University Press.Link
2. Olfati-Saber, R., Fax, J. A., & Murray, R. M., Consensus and Cooperation in Networked Multi-Agent Systems.Link
3. Java and JavaFX Official Documentation.Link

4. JavaFx Scene Builder Tutorial for beginners Youtube